

Policy as Code: OPA Guardrails With Fast Feedback



Published on May 7, 2023



Webstack
Builders

Table of Contents

Introduction	3
Understanding OPA and Rego	3
OPA Architecture	3
Rego Fundamentals	5
Common Rego Patterns	7
Policy Design Principles	9
Writing Effective Policies	9
Policy Organization	10
Testing Policies	13
Unit Testing Rego	13
Running Policy Tests	15
CI/CD Integration	17
Pre-commit Hooks	17
GitHub Actions Integration	18
Performance Optimization	21
Debugging Policy Failures	23
Terraform-Specific Policies	24
Terraform Plan Evaluation	24
HCL vs Plan JSON	27
Kubernetes-Specific Policies	28
Pod Security Policies	28
Resource and Image Policies	31
Severity Levels	33
Handling Exceptions	34
Exception Mechanisms	34
Exception Governance	37
Conclusion	38



Policy as Code: OPA Guardrails With Fast Feedback

Introduction

A developer opens a pull request for a Terraform change. Sixteen hours later, a security review rejects it: the S3 bucket lacks encryption. The developer fixes it, waits another day for re-review. This cycle – write, wait, reject, fix, wait – drains velocity and breeds resentment toward security processes.

Shift-left advocates say to check policies earlier. But “earlier” often means CI, which still means waiting for pipelines after pushing code. Real shift-left means *before the commit* – policy checks that run in seconds during `git add`, catching violations while the context is fresh and the fix is trivial.

This article covers building infrastructure guardrails with OPA (Open Policy Agent) (Open Policy Agent) and Conftest. The focus is on speed: pre-commit hooks that run in under two seconds, CI checks that parallelize across hundreds of policies, and feedback loops that make compliance a developer experience issue rather than a security bottleneck.

WARNING

Policy adoption paradox: comprehensive policies with slow feedback get disabled. Minimal policies with fast feedback get expanded. Start with five critical policies – encryption enabled, no public access, resource limits, no privileged containers, no `:latest` tags – that run fast. Get adoption first, then add coverage.

The target architecture runs Conftest against Terraform plans and Kubernetes manifests at three stages: pre-commit for immediate feedback, CI for enforcement, and deployment pipelines for final validation. Each stage uses the same policies but different optimization strategies.



Understanding OPA and Rego

OPA Architecture

OPA (Open Policy Agent) is a general-purpose policy engine that decouples policy decisions from policy enforcement. You feed it structured data (JSON), it evaluates policies written in Rego, and it returns decisions. The engine itself is stateless – policies and data define behavior.

Three deployment modes serve different use cases:

- 1 Library mode**
Embed OPA directly in applications for authorization decisions. Go applications can import OPA as a package; other languages use WASM or REST APIs.
- 2 Daemon mode**
Run OPA as a sidecar or standalone service. Kubernetes admission controllers use this pattern – the API server sends admission requests to OPA, which returns allow/deny decisions.
- 3 CLI mode**
Evaluate policies from the command line. This is where Conftest enters: it wraps OPA's CLI with ergonomic defaults for infrastructure files.

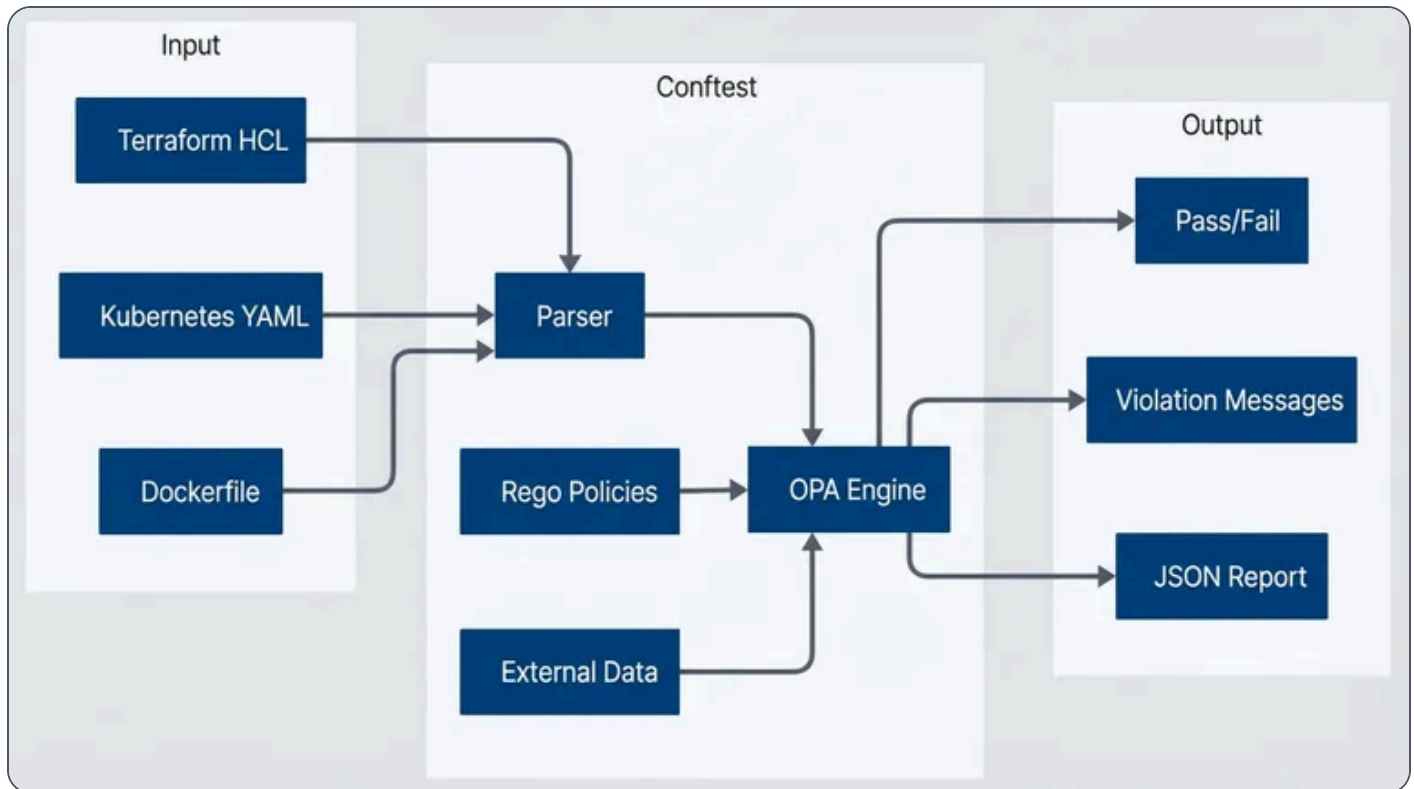
Conftest handles the plumbing that makes OPA (Open Policy Agent) practical for IaC (Infrastructure as Code). It parses Terraform, Kubernetes YAML, Dockerfiles, and other formats into JSON that OPA (Open Policy Agent) can evaluate. It provides sensible output formatting, exit codes for CI, and project conventions for organizing policies.

Bash

```
1  #!/bin/bash
2
3  # Basic Conftest usage
4  conftest test deployment.yaml --policy ./policies
5
```



```
6  # Test Terraform plan
7  terraform plan -out=tfplan
8  terraform show -json tfplan > tfplan.json
9  conftest test tfplan.json --policy ./policies/terraform
```



Conftest evaluation flow.

Rego Fundamentals

Rego is a declarative language designed for querying nested data structures. If you've used SQL, the mental model is similar: you describe *what* you want, not *how* to compute it. Unlike SQL, Rego operates on hierarchical JSON rather than relational tables.

Rules generate values. A rule with no conditions always produces its value. A rule with conditions only produces its value when all conditions succeed:



Rego

```
1  # Unconditional: always true
2  default allow := false
3
4  # Conditional: true only when conditions match
5  allow if {
6      input.user.role == "admin"
7      input.action == "read"
8  }
```

Rules with the same name combine. Multiple rules with the same head form a logical OR – any rule matching makes the head true:

Rego

```
1  # Either condition makes allow true
2  allow if { input.user.role == "admin" }
3  allow if { input.resource.public == true }
```

Iteration is implicit. Referencing an array index with a variable iterates over all elements:

Rego

```
1  # Finds ANY container without limits
2  deny contains msg if {
3      container := input.spec.containers[_] # Iterate all containers
4      not container.resources.limits
5      msg := sprintf("Container '%s' missing resource limits", [container.name])
6  }
```

Sets collect unique values. Using `contains` with a rule head builds a set of all values where conditions match:



Rego

```
1  # Collects all violations into a set
2  deny contains msg if {
3      # ... conditions ...
4      msg := "violation message"
5  }
```

External data enriches decisions. The `data` object provides context beyond the immediate input – allowed registries, team mappings, environment configurations:

Rego

```
1  # data.allowed_registries loaded from JSON file
2  deny contains msg if {
3      image := input.spec.containers[_].image
4      registry := split(image, "/")[0]
5      not registry in data.allowed_registries
6      msg := sprintf("Image '%s' from unauthorized registry", [image])
7  }
```

With these fundamentals in place – rules, iteration, sets, and data – you have the building blocks for any infrastructure policy. The patterns below show how these primitives combine for common validation scenarios.

Common Rego Patterns

#	Pattern	Use Case	Example
1	Required field	Ensure configuration exists	<code>not input.spec.selector</code>
2	Allowed values	Whitelist valid options	<code>input.type in {"ClusterIP", "NodePort"}</code>
3	Regex matching	Pattern validation	<code>regex.match("^[a-z][a-z0-9-]*\$", name)</code>



#	Pattern	Use Case	Example
4	Numeric constraints	Range validation	<code>input.replicas > 0; input.replicas <= 10</code>
5	Cross-resource	Reference related objects	<code>data.namespaces[input.metadata.namespace]</code>
6	Conditional	Environment-specific rules	<code>is_production; ... production-only rule</code> <code>...</code>

Common Rego validation patterns and representative examples.

The patterns most useful for infrastructure policies combine iteration with existence checks:

Rego

```

1  # Required labels pattern
2  required_labels := {"app", "team", "environment"}
3
4  deny contains msg if {
5      missing := required_labels - {label | input.metadata.labels[label]}
6      count(missing) > 0
7      msg := sprintf("Missing required labels: %v", [missing])
8  }
```

Required labels pattern using set difference.

The same structure applies to allowed values – define the valid set, check membership, and generate a message with the violation and valid options:

Rego

```

1  # Allowed values pattern
2  allowed_types := {"ClusterIP", "NodePort"}
3
4  deny contains msg if {
```




```
5     input.kind == "Service"
6     not input.spec.type in allowed_types
7     msg := sprintf("Service type '%s' not allowed. Use: %v", [input.spec.type,
allowed_types])
8 }
```

Allowed values pattern using set membership.

INFO

Rego's learning curve is real but bounded. Most infrastructure policies use a small subset of the language: iteration, existence checks, set operations, and string formatting. Master these patterns and you'll handle 90% of use cases.

Policy Design Principles

Writing Effective Policies

Four principles separate policies that get adopted from policies that get bypassed:

1

Single responsibility.

Each rule should check one thing. A 50-line rule that validates images, resources, and labels produces confusing violations and resists modification. Split concerns into separate rules: `deny_unauthorized_image`, `deny_missing_resources`, `deny_missing_labels`. When a violation fires, developers know exactly what to fix.

2

Actionable messages.

"Policy violation" tells the developer nothing. "Container missing resource limits" is better. "Container 'nginx' missing resource limits. Add `spec.containers[].resources.limits.cpu` and memory" is what they actually need. Include the resource name, the violation, and the fix path.



3

Minimal false positives.

Precision matters more than recall. A policy that blocks valid configurations trains developers to request exceptions — or bypass the system entirely. Start permissive and tighten over time. Test against real configurations before enforcing. Provide escape hatches for legitimate edge cases.

4

Fast evaluation.

Policies should evaluate in milliseconds, not seconds. Avoid external HTTP calls within policies — pre-load data into bundles. Avoid complex regex on large inputs. Avoid deep recursion. The goal is sub-second feedback that doesn't interrupt flow.

Structure violation messages consistently:

Markdown

```
1 [Resource Type] [Resource Name]: [Violation].
2 Fix: [Specific remediation]
```

Violation message template.

Examples:

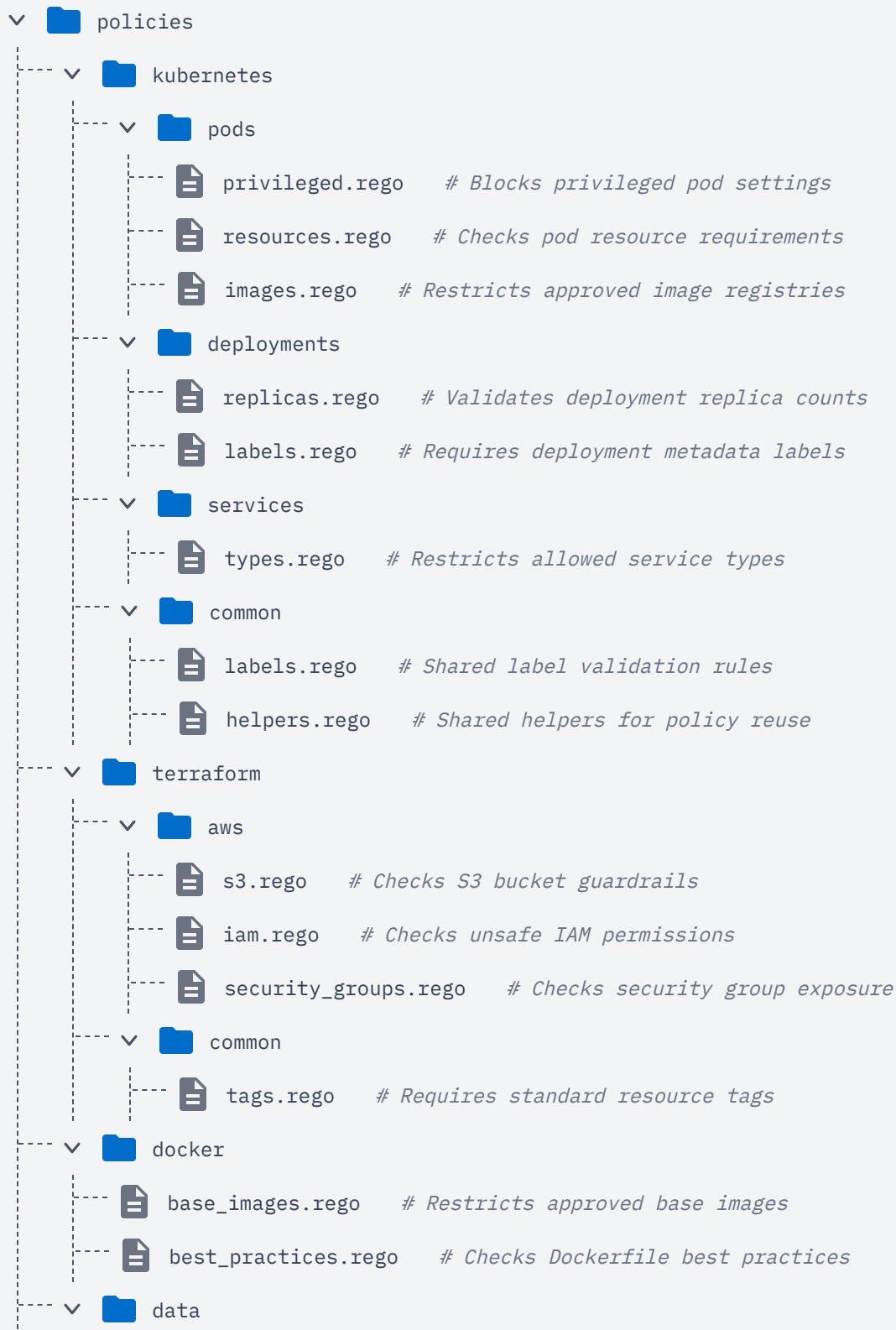
```
Deployment 'api-server': Missing required label 'team'. Fix:
Add metadata.labels.team with your team name.
```

```
Pod 'worker': Container 'app' uses image from unauthorized
registry 'docker.io'. Fix: Use images from 'gcr.io/company-
project' or 'artifactory.company.com'.
```

Policy Organization

Organize policies by technology, then resource type, then concern. This structure makes policies discoverable and enables selective evaluation — run only Terraform policies on Terraform files.







Policy directory structure.

Rego packages map to this directory structure. Use namespacing to group related policies and enable selective evaluation:

Rego

```

1  # Kubernetes pod security policies
2  package kubernetes.pods.security
3
4  deny contains msg if {
5      # Pod security checks
6  }
7
8  # Kubernetes pod image policies
9  package kubernetes.pods.images
10
11 deny contains msg if {
12     # Image validation checks
13 }
14
15 # Terraform AWS S3 policies
16 package terraform.aws.s3
17
18 deny contains msg if {
19     # S3 bucket checks
20 }
21
22 # Shared helper library
  
```



```
23 package lib.helpers
24
25 is_kubernetes_resource if {
26     input.apiVersion
27     input.kind
28 }
29
30 has_label(obj, label) if {
31     obj.metadata.labels[label]
32 }
```

Policy namespaces by package.

Conftest uses the `--namespace` flag to selectively evaluate policies. This is how namespace organization pays off – you can run subsets of policies based on context:

Bash

```
1 # Run only Kubernetes policies on YAML files
2 conftest test deployment.yaml --namespace kubernetes
3
4 # Run only pod-specific policies
5 conftest test deployment.yaml --namespace kubernetes.pods
6
7 # Run all sub-namespaces (security, images, resources)
8 conftest test deployment.yaml --namespace kubernetes.pods --all-namespaces
9
10 # Run only Terraform AWS policies
11 conftest test tfplan.json --namespace terraform.aws
12
13 # Run all policies (default behavior)
14 conftest test deployment.yaml
```

Selective policy evaluation with namespaces.

This enables layered enforcement: pre-commit runs fast, critical policies; CI runs comprehensive checks; deployment gates run everything. Same policies, different namespace filters.



✓ SUCCESS

Put shared helper functions in `lib/helpers.rego` and import them with `import data.lib.helpers`. This avoids duplicating normalization logic across policies and makes updates propagate automatically.

Testing Policies

Unit Testing Rego

OPA (Open Policy Agent) includes a built-in test framework. Test files use `_test` suffix in the package name and define rules prefixed with `test_`. Each test rule should evaluate to `true` when the test passes.

The `with` keyword is the key to Rego testing – it lets you override `input` and `data` for a specific evaluation:

Rego

```
1 package kubernetes.pods.resources_test
2
3 import future.keywords.if
4 import data.kubernetes.pods.resources
5
6 # Test that valid config passes (no violations)
7 test_pod_with_limits if {
8     count(resources.deny) == 0 with input as {
9         "kind": "Pod",
10        "metadata": {"name": "good-pod"},
11        "spec": {
12            "containers": [{
13                "name": "app",
14                "image": "nginx:1.21",
15                "resources": {
16                    "limits": {"cpu": "100m", "memory": "128Mi"},
17                    "requests": {"cpu": "50m", "memory": "64Mi"}
18                }
19            }
20        }
21    }
```



```

19         }]
20     }
21 }
22 }
23
24 # Test that invalid config fails (has violations)
25 test_pod_without_limits if {
26     violations := resources.deny with input as {
27         "kind": "Pod",
28         "metadata": {"name": "bad-pod"},
29         "spec": {
30             "containers": [{
31                 "name": "app",
32                 "image": "nginx:1.21"
33             }]
34         }
35     }
36     count(violations) > 0
37     violation := violations[_]
38     contains(violation, "missing resource limits")
39 }
40
41 # Test edge case: multiple containers, one without limits
42 test_multiple_containers_partial if {
43     violations := resources.deny with input as {
44         "kind": "Pod",
45         "spec": {
46             "containers": [
47                 {"name": "good", "resources": {"limits": {"cpu": "100m"}}},
48                 {"name": "bad"}
49             ]
50         }
51     }
52     count(violations) == 1
53     contains(violations[_], "bad")
54 }
55
56 # Test with external data override
57 test_allowed_registry if {
58     count(resources.deny) == 0 with input as {
59         "kind": "Pod",
60         "spec": {

```



```

61         "containers": [{
62             "name": "app",
63             "image": "gcr.io/company/app:v1.0.0"
64         }]
65     }
66     } with data.allowed_registries as ["gcr.io/company"]
67 }
```

Rego unit tests with input and data overrides.

Running Policy Tests

OPA (Open Policy Agent)'s test runner provides verbose output, coverage reporting, and filtering:

Bash

```

1  # Run all tests in policies directory
2  opa test policies/ --verbose
3
4  # Output:
5  # data.kubernetes.pods.resources_test.test_pod_with_limits: PASS (1.234ms)
6  # data.kubernetes.pods.resources_test.test_pod_without_limits: PASS (0.567ms)
7  # PASS: 15/15
8
9  # Run tests with coverage reporting
10 opa test policies/ --coverage --format=json > coverage.json
11
12 # Enforce minimum coverage threshold
13 opa test policies/ --coverage --threshold 80
14
15 # Run specific test file
16 opa test policies/kubernetes/pods/resources_test.rego --verbose
17
18 # Run tests matching pattern
19 opa test policies/ --verbose --run "test_.*registry.*"
20
21 # Benchmark test performance
22 opa test policies/ --bench
```

OPA (Open Policy Agent) test runner commands.



For integration testing against real YAML/JSON fixtures, Conftest provides `verify` mode:

Bash

```
1  # Test that valid configs pass
2  conftest verify --policy policies/ test/fixtures/valid_pod.yaml
3
4  # Test that invalid configs fail (expect exit code 1)
5  conftest verify --policy policies/ test/fixtures/invalid_pod.yaml || echo "Expected
   failure"
6
7  # Run all Rego unit tests via Conftest
8  conftest verify --policy policies/
```

Conftest verification commands.

⚠ WARNING

Test both positive cases (valid configs pass) and negative cases (invalid configs fail). A policy that blocks everything is as broken as one that allows everything.

CI/CD Integration

Pre-commit Hooks

Pre-commit is where shift-left becomes real. Conftest integrates with the pre-commit framework to run policies against staged files before they're committed. The key is speed – pre-commit hooks that take more than a few seconds get disabled.

YAML

```
1  # .pre-commit-config.yaml
2  repos:
3    - repo: https://github.com/open-policy-agent/conftest
4      rev: v0.45.0
```



```

5     hooks:
6       - id: conftest
7         name: Conftest Kubernetes
8         entry: conftest test
9         args:
10          - --policy=policies/kubernetes
11          - --all-namespaces
12         files: '\.ya?ml$'
13         exclude: 'policies/.*'
14
15       - id: conftest
16         name: Conftest Terraform
17         entry: conftest test
18         args:
19          - --policy=policies/terraform
20          - --parser=hcl2
21         files: '\.tf$'
22
23       - id: conftest
24         name: Conftest Dockerfile
25         entry: conftest test
26         args:
27          - --policy=policies/docker
28          - --parser=dockerfile
29         files: 'Dockerfile'

```

Pre-commit configuration for Conftest.

Pre-commit only runs against staged files by default, which keeps evaluation fast. Separate hooks for different file types ensure Kubernetes policies don't run on Terraform files and vice versa.

The pre-commit framework <<https://pre-commit.com/>> shown above is language-agnostic and works across ecosystems, but each stack has its canonical approach:

Ecosystem	Pre-commit Tool	Installation
Python	pre-commit	<code>pip install pre-commit && pre-commit install</code>
Node.js	Husky + lint-staged	<code>npx husky init</code>



Ecosystem	Pre-commit Tool	Installation
Ruby/Rails	Overcommit	<code>gem install overcommit && overcommit --install</code>
PHP/Laravel	GrumPHP	<code>composer require --dev phpro/grumphp</code>
.NET/C#	Husky.Net	<code>dotnet tool install husky</code>
Go	pre-commit or lefthook	<code>go install github.com/evilmartians/lefthook</code>

Pre-commit tools by ecosystem.

Husky is Node.js-based but runs any shell command, making it viable for polyglot repositories with a `package.json`. For pure infrastructure repos without Node, the Python pre-commit framework or Lefthook (Go-based, no runtime dependencies) are better choices.

GitHub Actions Integration

CI provides the enforcement layer – policies that developers can't bypass. The workflow runs policy tests first (catching broken policies), then evaluates Kubernetes and Terraform configurations in parallel.

YAML

```

1  # .github/workflows/policy-check.yml
2  name: Policy Check
3
4  on:
5    pull_request:
6      paths:
7        - '**/*.yaml'
8        - '**/*.yml'
9        - '**/*.tf'
10       - 'Dockerfile*'
11       - 'policies/**'
12
13  jobs:
14    policy-test:
15      name: Test Policies
16      runs-on: ubuntu-latest

```



```

17     steps:
18       - uses: actions/checkout@v4
19       - name: Setup OPA
20         uses: open-policy-agent/setup-opa@v2
21       with:
22         version: latest
23       - name: Run policy tests
24         run: opa test policies/ --verbose --coverage --threshold 80
25
26   kubernetes-policies:
27     name: Kubernetes Policy Check
28     runs-on: ubuntu-latest
29     steps:
30       - uses: actions/checkout@v4
31       - name: Setup Conftest
32         uses: instrumenta/conftest-action@master
33       - name: Find Kubernetes files
34         id: k8s-files
35       run: |
36         FILES=$(find . -name '*.yaml' -o -name '*.yml' | grep -v policies/ | xargs)
37         echo "files=$FILES" >> $GITHUB_OUTPUT
38       - name: Run Conftest
39       run: |
40         conftest test ${ steps.k8s-files.outputs.files } \
41           --policy policies/kubernetes \
42           --all-namespaces \
43           --output github
44
45   terraform-policies:
46     name: Terraform Policy Check
47     runs-on: ubuntu-latest
48     steps:
49       - uses: actions/checkout@v4
50       - name: Setup Conftest
51         uses: instrumenta/conftest-action@master
52       - name: Setup Terraform
53         uses: hashicorp/setup-terraform@v3
54       - name: Terraform init and plan
55       run: |
56         terraform init
57         terraform plan -out=tfplan
58         terraform show -json tfplan > tfplan.json

```



```

59     - name: Run Conftest on plan
60       run: |
61         conftest test tfplan.json \
62           --policy policies/terraform \
63           --output github

```

GitHub Actions workflow for policy checks.

The `--output github` flag formats violations as GitHub annotations, surfacing them directly in the PR (Pull Request) diff where developers see them.

Branch protection is required for enforcement. Without it, developers can merge directly to main, bypassing CI entirely. In GitHub, navigate to Settings → Branches → Add rule, then enable “Require status checks to pass before merging” and select your policy jobs. This is a manual step that’s easy to forget when setting up new repositories.

Other platforms have equivalent mechanisms:

Platform	Protection Mechanism	Configuration Location
GitHub	Branch protection rules	Settings → Branches → Add rule
GitLab	Protected branches + merge request approvals	Settings → Repository → Protected branches
Bitbucket	Branch permissions + merge checks	Repository settings → Branch permissions
Azure DevOps	Branch policies	Repos → Branches → Branch policies
AWS CodePipeline	Approval actions + CodeBuild gates	Pipeline definition (no native branch protection)

Branch protection across CI platforms.

AWS CodePipeline lacks native branch protection – you’ll need to combine it with a Git provider’s branch rules or add manual approval stages. Azure DevOps and GitLab offer the most granular controls, including path-based policies and approval requirements.



Performance Optimization

Policy checks that exceed 30 seconds get skipped “just this once”—and then every time after. Four optimizations keep evaluation fast:

Namespace filtering. Only evaluate relevant policies. Kubernetes policies for YAML files, Terraform policies for plan JSON:

Bash

```
1  conftest test k8s/*.yaml --namespace kubernetes
2  conftest test tfplan.json --namespace terraform.aws
```

Namespace filtering for faster evaluation.

Parallel execution. Evaluate multiple files simultaneously:

Bash

```
1  find . -name '*.yaml' -print0 | xargs -0 -P4 -n10 conftest test --policy policies/
```

Parallel file evaluation with xargs.

Policy bundling. Pre-compile policies into optimized bundles:

Bash

```
1  # Build optimized bundle
2  opa build -b policies/ -o bundle.tar.gz
3
4  # Use bundle in evaluation
5  conftest test --policy bundle.tar.gz input.yaml
```

Pre-compiled policy bundles.

CI caching. Cache policy bundles between runs:



YAML

```

1 - uses: actions/cache@v3
2   with:
3     path: ~/.cache/conftest
4     key: conftest-policies-${{ hashFiles('policies/**') }}

```

GitHub Actions cache configuration.

Optimization	Speedup	Example (50 policies, 100 files)	Complexity
Namespace filtering	2-4x	8s → 2s	Low
Parallel execution	2-4x	8s → 2s	Low
Policy bundling	1.5-2x	8s → 5s	Medium
Selective file patterns	2-5x	8s → 2s	Low
Combined	5-15x	8s → 0.6s	Medium

Performance optimization impact.

Debugging Policy Failures

When policies produce unexpected results, OPA (Open Policy Agent)'s tracing and printing capabilities help identify the problem.

Print debugging. Add `print()` statements to see intermediate values:

Rego

```

1 deny contains msg if {

```



```
2     container := containers[_]
3     print("Checking container:", container.name, "image:", container.image)
4     endswith(container.image, ":latest")
5     msg := sprintf("Container '%s' uses :latest", [container.name])
6 }
```

Debug output with print statements.

Trace evaluation. OPA (Open Policy Agent)'s `--explain` flag shows the evaluation path:

Bash

```
1 # Show full evaluation trace
2 opa eval --data policies/ --input input.json "data.kubernetes.security.deny" --explain
  full
3
4 # Show only failed evaluations
5 opa eval --data policies/ --input input.json "data.kubernetes.security.deny" --explain
  fails
```

OPA (Open Policy Agent) evaluation tracing.

Parse input. Verify Conftest parses your file correctly:

Bash

```
1 # See what Conftest sees
2 conftest parse deployment.yaml --output json | jq .
3
4 # Test against parsed output directly
5 conftest parse deployment.yaml --output json > parsed.json
6 opa eval --data policies/ --input parsed.json "data.kubernetes.security.deny"
```

Inspecting parsed input.



✓ SUCCESS

The 30-second rule: if policy checks take longer than 30 seconds, developers will find ways to skip them. Invest in performance optimization to keep policies in the fast feedback loop.

Terraform-Specific Policies

Terraform Plan Evaluation

Terraform plan JSON contains the full resolved state of what Terraform **will** create – variables interpolated, modules expanded, data sources resolved. This is where security policies have complete visibility.

The plan JSON structure nests resource changes under `input.resource_changes`. Each change includes `before` (current state), `after` (planned state), and `actions` (create, update, delete, no-op). Helper functions make policies more readable:

Rego

```
1  package terraform.aws
2
3  import future.keywords.if
4  import future.keywords.in
5  import future.keywords.contains
6
7  # Helper: get all resources being created or modified
8  resources[resource] {
9      resource := input.resource_changes[_]
10     resource.change.actions[_] != "no-op"
11 }
12
13 # Helper: filter resources by type
14 resources_by_type(type) := [r | r := resources[_]; r.type == type]
```

Terraform plan helper functions.

With helpers in place, policies become concise. S3 bucket security:



Rego

```

1  deny contains msg if {
2      bucket := resources_by_type("aws_s3_bucket")[_]
3      bucket.change.after.acl == "public-read"
4      msg := sprintf("S3 bucket '%s' cannot be public-read", [bucket.name])
5  }
6
7  deny contains msg if {
8      bucket := resources_by_type("aws_s3_bucket")[_]
9      not bucket.change.after.server_side_encryption_configuration
10     msg := sprintf("S3 bucket '%s' must have encryption enabled", [bucket.name])
11 }

```

S3 bucket policies.

Security group rules that block open SSH/RDP:

Rego

```

1  deny contains msg if {
2      sg := resources_by_type("aws_security_group")[_]
3      rule := sg.change.after.ingress[_]
4      rule.cidr_blocks[_] == "0.0.0.0/0"
5      rule.from_port <= 22
6      rule.to_port >= 22
7      msg := sprintf("Security group '%s' allows SSH from 0.0.0.0/0", [sg.name])
8  }
9
10 deny contains msg if {
11     sg := resources_by_type("aws_security_group")[_]
12     rule := sg.change.after.ingress[_]
13     rule.cidr_blocks[_] == "0.0.0.0/0"
14     rule.from_port <= 3389
15     rule.to_port >= 3389
16     msg := sprintf("Security group '%s' allows RDP from 0.0.0.0/0", [sg.name])
17 }

```

Security group policies blocking open access.

IAM policies that prevent wildcard admin access:



Rego

```
1 deny contains msg if {
2     policy := resources_by_type("aws_iam_policy")[_]
3     statement := json.unmarshal(policy.change.after.policy).Statement[_]
4     statement.Effect == "Allow"
5     statement.Action[_] == "*"
6     statement.Resource == "*"
7     msg := sprintf("IAM policy '%s' grants admin access (Action:* Resource:*)",
8         [policy.name])
9 }
```

IAM policy preventing wildcard permissions.

Tagging requirements that apply across resource types:

Rego

```
1 required_tags := {"Environment", "Team", "CostCenter"}
2
3 deny contains msg if {
4     resource := resources[_]
5     resource.change.after.tags != null
6     provided := {tag | resource.change.after.tags[tag]}
7     missing := required_tags - provided
8     count(missing) > 0
9     msg := sprintf("Resource '%s' (%s) missing required tags: %v",
10         [resource.name, resource.type, missing])
11 }
```

Required tagging policy.

HCL vs Plan JSON

Two approaches exist for evaluating Terraform configurations, each with tradeoffs. HCL (HashiCorp Configuration Language) is Terraform's native syntax – the `.tf` files developers write. Plan JSON is the output of `terraform show -json`, containing the fully resolved execution plan with all variables interpolated and modules expanded.



Aspect	HCL Parsing	Plan JSON
Speed	Fast (< 1 second)	Slow (requires terraform plan)
Credentials	None required	Cloud credentials needed
Variable resolution	No	Yes
Module expansion	No	Yes
Data source values	No	Yes

HCL vs Plan JSON comparison.

HCL parsing is best for syntax checks and naming conventions where speed matters. Plan JSON is best for security policies and compliance checks that need the complete resolved configuration.

HCL parsing (`conftest test *.tf --parser hc12`) catches static violations without cloud access. Plan JSON evaluation sees the complete resolved configuration. Use both at different stages:

Stage	Check	Time
Developer machine	HCL static analysis	< 5 seconds
PR opened	HCL + basic plan	< 2 minutes
PR approved	Full plan analysis	< 5 minutes

Layered evaluation stages.

INFO

Use HCL parsing for fast feedback (pre-commit, early CI) and plan JSON for comprehensive checks (late CI, pre-deploy). The two approaches complement each other.



Kubernetes-Specific Policies

Pod Security Policies

Kubernetes manifests present a challenge: the same pod spec appears in Pods, Deployments, DaemonSets, StatefulSets, Jobs, and CronJobs – each at a different path in the YAML structure. Effective policies need helper functions to normalize access.

Conftest evaluates these policies during CI/CD, but you can also enforce them at runtime using OPA (Open Policy Agent) Gatekeeper <<https://open-policy-agent.github.io/gatekeeper/website/>> – a Kubernetes admission controller that rejects non-compliant resources at apply time. The Rego logic is identical; Gatekeeper just runs it as a webhook instead of a CI check.

Rego

```
1  package kubernetes.security
2
3  import future.keywords.if
4  import future.keywords.in
5  import future.keywords.contains
6
7  # Normalize pod spec access across resource types
8  is_pod_or_controller if { input.kind == "Pod" }
9  is_pod_or_controller if { input.kind in {"Deployment", "DaemonSet", "StatefulSet",
10 "ReplicaSet", "Job", "CronJob"} }
11
12 pod_spec := input.spec if { input.kind == "Pod" }
13 pod_spec := input.spec.template.spec if { input.kind in {"Deployment", "DaemonSet",
14 "StatefulSet", "ReplicaSet", "Job"} }
15 pod_spec := input.spec.jobTemplate.spec.template.spec if { input.kind == "CronJob" }
16
17 # Include init containers in checks
18 containers := pod_spec.containers
19 containers := array.concat(pod_spec.containers, pod_spec.initContainers) if {
20 pod_spec.initContainers }
```

Helper functions for Kubernetes resource normalization.

With normalization in place, security policies become straightforward:



Rego

```

1  # Block privileged containers
2  deny contains msg if {
3      is_pod_or_controller
4      container := containers[_]
5      container.securityContext.privileged == true
6      msg := sprintf("Container '%s' must not run privileged", [container.name])
7  }
8
9  # Block root user
10 deny contains msg if {
11     is_pod_or_controller
12     container := containers[_]
13     container.securityContext.runAsUser == 0
14     msg := sprintf("Container '%s' must not run as root (UID 0)", [container.name])
15 }
16
17 # Require read-only root filesystem
18 deny contains msg if {
19     is_pod_or_controller
20     container := containers[_]
21     not container.securityContext.readOnlyRootFilesystem == true
22     msg := sprintf("Container '%s' should use read-only root filesystem",
23         [container.name])
24 }
25
26 # Block privilege escalation
27 deny contains msg if {
28     is_pod_or_controller
29     container := containers[_]
30     container.securityContext.allowPrivilegeEscalation == true
31     msg := sprintf("Container '%s' must not allow privilege escalation",
32         [container.name])
33 }

```

Container security context policies.

Capability and host namespace restrictions prevent container escape vectors:



Rego

```

1  dangerous_capabilities := {"NET_ADMIN", "SYS_ADMIN", "ALL"}
2
3  deny contains msg if {
4      is_pod_or_controller
5      container := containers[_]
6      cap := container.securityContext.capabilities.add[_]
7      cap in dangerous_capabilities
8      msg := sprintf("Container '%s' adds dangerous capability: %s", [container.name,
    cap])
9  }
10
11 deny contains msg if {
12     is_pod_or_controller
13     pod_spec.hostNetwork == true
14     msg := "Pod must not use host network"
15 }
16
17 deny contains msg if {
18     is_pod_or_controller
19     pod_spec.hostPID == true
20     msg := "Pod must not use host PID namespace"
21 }

```

Capability and host namespace restrictions.

Resource and Image Policies

Resource limits prevent noisy neighbor problems; image policies prevent supply chain attacks. The severity differs by environment – production requires limits, development can warn:

Rego

```

1  package kubernetes.resources
2
3  import future.keywords.if
4  import future.keywords.contains
5
6  # Environment detection helper

```



```

7  is_production if { input.metadata.labels.environment == "production" }
8  is_production if { input.metadata.labels.env == "prod" }
9  is_production if { startswith(input.metadata.namespace, "prod") }
10
11 # Production requires resource limits
12 deny contains msg if {
13     is_production
14     container := containers[_]
15     not container.resources.limits.cpu
16     msg := sprintf("Container '%s' missing CPU limit (required in production)",
17 [container.name])
18 }
19 deny contains msg if {
20     is_production
21     container := containers[_]
22     not container.resources.limits.memory
23     msg := sprintf("Container '%s' missing memory limit (required in production)",
24 [container.name])
25 }
26 # Non-production gets warnings
27 warn contains msg if {
28     not is_production
29     container := containers[_]
30     not container.resources.requests
31     msg := sprintf("Container '%s' should have resource requests", [container.name])
32 }

```

Environment-specific resource policies.

Image policies enforce versioning and registry restrictions:

Rego

```

1  package kubernetes.images
2
3  import future.keywords.if
4  import future.keywords.contains
5
6  # Block :latest tag

```




```
7  deny contains msg if {
8      container := containers[_]
9      endswith(container.image, ":latest")
10     msg := sprintf("Container '%s' uses 'latest' tag. Use specific version",
11         [container.name])
12 }
13 # Require explicit tag
14 deny contains msg if {
15     container := containers[_]
16     not contains(container.image, ":")
17     msg := sprintf("Container '%s' has no tag. Always specify image tag",
18         [container.name])
19 }
20 # Restrict to allowed registries
21 allowed_registries := data.allowed_registries
22
23 deny contains msg if {
24     count(allowed_registries) > 0
25     container := containers[_]
26     image := container.image
27     not any_registry_allowed(image)
28     msg := sprintf("Image '%s' not from allowed registry. Allowed: %v", [image,
29         allowed_registries])
30 }
31 any_registry_allowed(image) if {
32     registry := allowed_registries[_]
33     startswith(image, registry)
34 }
```

Image tag and registry policies.

Severity Levels

Conftest supports three severity levels: `deny` (blocks deployment), `warn` (allows with warning), and `violation` (informational). Map severity to policy type and environment:



#	Policy Type	Production	Non-Production
1	Security (privileged, root)	deny	deny
2	Resource limits	deny	warn
3	Image tags	deny	warn
4	Naming conventions	warn	info

Severity mapping by environment.

A single policy file can define both `deny` and `warn` rules. Conftest evaluates all rules and reports violations by severity— `deny` violations cause a non-zero exit code, `warn` violations are reported but don't block:

Rego

```

1  package kubernetes.images
2
3  import future.keywords.if
4  import future.keywords.contains
5
6  # Hard block: :latest is never acceptable
7  deny contains msg if {
8      container := containers[_]
9      endswith(container.image, ":latest")
10     msg := sprintf("Container '%s' uses :latest tag - blocked", [container.name])
11 }
12
13 # Soft warning: prefer digests over tags for supply chain security
14 warn contains msg if {
15     container := containers[_]
16     not contains(container.image, "@sha256:")
17     msg := sprintf("Container '%s' uses tag instead of digest - consider pinning",
18         [container.name])
19 }
```

Combining deny and warn in a single policy file.



⚠ WARNING

Use `warn` for best practices and `deny` for security requirements. Too many `deny` rules create friction; too few create risk. Start with warnings, promote to denials based on adoption.

Handling Exceptions

Policies without escape hatches get bypassed entirely. Legitimate exceptions exist – CNI plugins need privileged access, node exporters need host network, legacy applications need time to migrate. Effective exception handling isn't about eliminating workarounds – it's about governing them with justification, expiration, and audit trails.

Exception Mechanisms

Three patterns handle exceptions at different granularities. Choose based on your governance model: annotation-based for developer self-service, expiring exceptions for temporary workarounds, and data-driven for centralized management.

Annotation-based exceptions let developers request exceptions inline with the resource. The policy checks for an exception annotation before denying:

Rego

```
1  package kubernetes.security
2
3  import future.keywords.if
4  import future.keywords.contains
5
6  deny contains msg if {
7      container := containers[_]
8      container.securityContext.privileged == true
9      not has_exception("privileged")
10     msg := sprintf("Container '%s' runs privileged without exception", [container.name])
11 }
12
13 has_exception(policy_name) if {
```



```

14     annotation := sprintf("policy.company.com/exception-%s", [policy_name])
15     input.metadata.annotations[annotation]
16 }

```

Annotation-based exception checking.

Expiring exceptions prevent permanent workarounds. The policy parses an expiration date and blocks if the exception has lapsed:

Rego

```

1  deny contains msg if {
2      container := containers[_]
3      container.securityContext.privileged == true
4      exception_expired("privileged")
5      msg := sprintf("Container '%s' privileged exception has expired", [container.name])
6  }
7
8  exception_expired(policy_name) if {
9      annotation := sprintf("policy.company.com/exception-%s-expires", [policy_name])
10     expiry := input.metadata.annotations[annotation]
11     time.parse_rfc3339_ns(expiry) < time.now_ns()
12 }

```

Time-limited exception with expiration.

Data-driven exceptions centralize exception management. Instead of annotations scattered across manifests, a JSON file defines which resources are excepted:

JSON

```

1  {
2      "privileged": [
3          {"namespace": "kube-system", "name": "cni-*"},
4          {"namespace": "monitoring", "name": "node-exporter"}
5      ]
6  }

```

Centralized exception data file.



Rego

```
1  is_excepted(policy_name) if {
2      exception := data.exceptions[policy_name][_]
3      exception.namespace == input.metadata.namespace
4      glob.match(exception.name, [], input.metadata.name)
5  }
6
7  deny contains msg if {
8      container := containers[_]
9      container.securityContext.privileged == true
10     not is_excepted("privileged")
11     msg := sprintf("Container '%s' runs privileged", [container.name])
12 }
```

Data-driven exception lookup with glob matching.

Exception Governance

Exceptions without governance accumulate indefinitely. A structured process keeps exceptions visible and time-bounded:

1

Developer adds exception annotation with reason

2

PR triggers policy check – warns about exception usage

3

Security team reviews exception request

4

If approved, merge with exception and expiration date



5 Exception logged in audit system

The annotation format captures required metadata:

YAML

```
1 metadata:
2   annotations:
3     policy.company.com/exception-privileged: "true"
4     policy.company.com/exception-reason: "CNI plugin requires network namespace access"
5     policy.company.com/exception-expires: "2024-06-01"
6     policy.company.com/exception-owner: "team-networking"
```

Exception annotation format.

Different exception types warrant different governance levels:

Exception Type	Max Duration	Review Frequency	Approval Required
Privileged container	30 days	Weekly	Security team
Host network	30 days	Weekly	Security team
Missing limits	90 days	Monthly	Team lead
Non-standard image	90 days	Monthly	Team lead
Deprecated API	180 days	Quarterly	Platform team

Exception governance by type.



Track exception metrics to identify patterns: total active exceptions, exceptions by policy type, exceptions by team, expired exceptions still in use, average exception age. Alert on anomalies – exceptions granted without expiration, exceptions older than 90 days, namespaces with excessive exceptions.

INFO

Exceptions are a feature, not a bug. Legitimate use cases exist for most security rules. The goal is governance: require justification, set expiration, track usage, and review regularly.

Conclusion

Effective infrastructure guardrails require more than comprehensive policies – they require fast feedback, clear messages, appropriate severity levels, and pragmatic exception handling. Policies that run in pre-commit catch violations before they become PRs. Policies with actionable messages get fixed instead of ignored. Policies with exception mechanisms accommodate legitimate edge cases.

SUCCESS

Start with five critical policies that run in under two seconds. Get adoption. Add coverage. The fastest path to comprehensive guardrails runs through developer trust.

The goal is policies that developers trust: fast enough to not slow them down, accurate enough to not cry wolf, and flexible enough to handle real-world complexity. The measure of success isn't how many violations you block – it's how few violations reach production combined with how little friction developers experience. Both matter.



Copyright © 2023 Webstack Builders, Inc.

The text, diagrams, and images in this work are licensed under CC BY-NC 4.0

All code samples in this article are licensed under the MIT License. Feel free to use, modify, and distribute them in any project.

<https://www.webstackbuilders.com/articles/opa-conftest-policy-as-code-infrastructure-guardrails>

